



Islamic University of Technology
Department of Computer Science and Engineering

Lab 9: Divide and Conquer

CSE 4404: Algorithms Lab
Summer 2024-25

Contents

Task A. Divide and Conquer Convex Hull **2**

Task Statement 2

Input Format 2

Output Format 2

Constraints 2

Examples 3

Notes 3

Task B. Maximum Subarray Sum **5**

Task Statement 5

Input Format 5

Output Format 5

Constraints 5

Examples 5

Notes 5

Task C. Longest Nice Substring **6**

Task Statement 6

Input Format 6

Output Format 6

Constraints 6

Examples 6

Notes 6

Task A. Divide and Conquer Convex Hull

Time Limit: 2 seconds | Memory Limit: 1536 MB

Given a set of N distinct points on a two-dimensional plane where no two points share identical X or Y coordinates, your objective is to find the vertices that form the **Convex Hull** of these points in a strict **Clockwise (CW)** order.

To achieve optimal performance constraints, you are required to implement the **Preparata–Hong Divide and Conquer** algorithm:

1. Sort all points globally by their X -coordinate (breaking ties with Y -coordinates).
2. Recursively divide the sorted set into left and right halves.
3. For base cases containing 5 or fewer points (≤ 5), compute the convex hull directly using a **Brute-Force** method, ensuring the returned subset is oriented clockwise.
4. Merge the two independent clockwise sub-hulls back together using the **two-finger tangent walking method** to find both the upper and lower common tangents, subsequently applying a **cut-and-paste** step to stitch the outer boundary cleanly.

Output Sequence Requirement

The output configuration requires specific formatting to be accepted by the structural test judge:

- The convex hull vertices must be printed as a single space-separated sequence of their original, 1-based input indices.
- The sequence must follow a continuous **Clockwise** perimeter.
- **Crucial Boundary Condition:** The output cycle must be cyclically shifted/rotated so that the point with the earliest original input index (the point that appeared first in the input list, which bears the minimum Tracking ID) is positioned at the very beginning of the output sequence.

Input Format

The first line contains a single integer N , representing the total number of points on the plane. The next N lines each contain two space-separated integers x_i and y_i , denoting the coordinate plane positions of the i -th point.

Output Format

Print a single line containing the original 1-based indices of the points that form the convex hull, ordered clockwise and beginning with the earliest input index point.

Constraints

- $3 \leq N \leq 10^5$
- $-10^9 \leq x_i, y_i \leq 10^9$
- All x_i coordinates are distinct.
- All y_i coordinates are distinct.

Examples

Sample Input	Sample Output
6 1 4 2 8 4 1 5 5 7 9 9 3	1 2 5 6 3

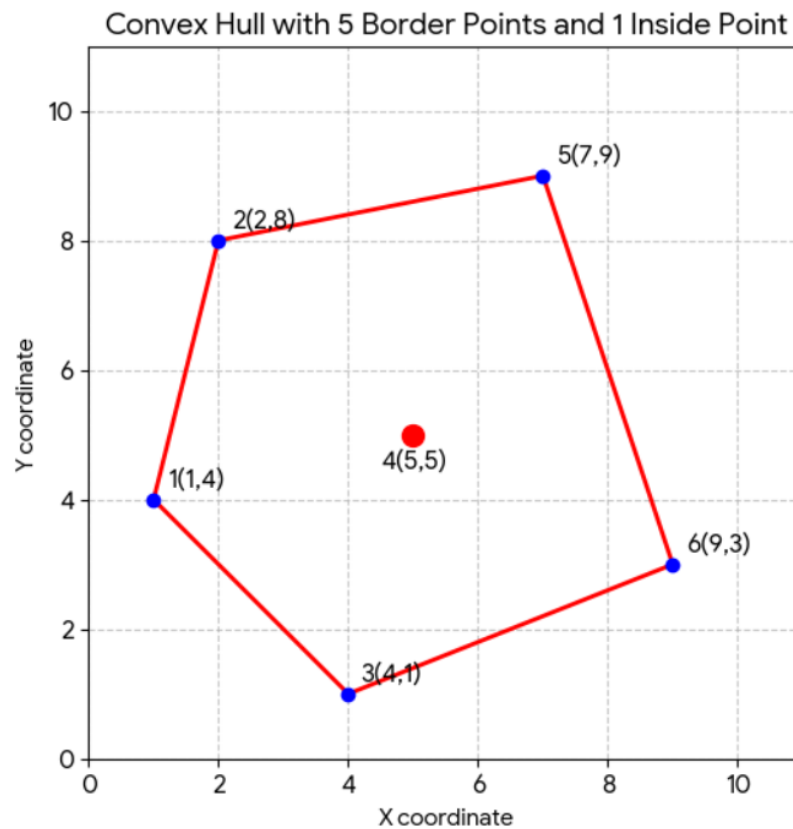


Figure 1: Visual representation of the sample points and their clockwise convex hull.

Notes

A skeleton .cpp file is provided in the submission folder, where most functions are already implemented. You have to fill up the missing part of the code. Descriptions of some functions are given below:

1. **comparePoints**: Used to sort the points using x values.
2. **isValid**: Checks if all the points fall on the right side of the current line or not.
3. **getYIntersection**: Finds the y value of the intersection of the vertical line and the line formed by connecting two points from both sides.

4. `findUpperTangent`: Finds the two points forming the upper tangent.
5. `combine`: Returns the sequence of points after combining two polygons, given the upper and lower tangent points.
6. `mergeHulls`: Uses all the functions above to merge two convex polygons.

Task B. Maximum Subarray Sum

Time Limit: 1 second | Memory Limit: 512 MB

You are given an array of n integers. Your task is to find the maximum sum of a subarray. A subarray is a contiguous part of the array.

Input Format

The first input line has an integer n : the size of the array. The next line has n integers $x_1, x_2, \dots, x_n$: the contents of the array.

Output Format

Print one integer: the maximum subarray sum.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $-10^9 \leq x_i \leq 10^9$

Examples

Sample Input	Sample Output
8 -1 3 -2 5 3 -5 2 2	9

Notes

You must use a divide and conquer approach.

Task C. Longest Nice Substring

Time Limit: 1 second | Memory Limit: 512 MB

A string s is **nice** if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase. For example, "aAa" is nice because both "A" and "a" appear, but "abA" is not because "b" appears but "B" does not.

Given a string s , return the longest substring of s that is nice. If there are multiple, return the substring that occurs first in the input string.

Input Format

The input will contain a single line containing a string with both uppercase and lowercase alphabets.

Output Format

Print one string: representing the longest nice substring.

Constraints

- $1 \leq s.length \leq 100$
- s consists of uppercase and lowercase English letters.

Examples

Sample Input	Sample Output
YazaAay	aAa
Bb	Bb
c	

Notes

You must use a divide and conquer approach.

Marks Distribution

Task	Marks
Task A	40%
Task B	30%
Task C	30%

Practice Problems

Problem	Link
---------	------